Modulo 6

Serverless y Lambda

Curso: AWS Cloud Practitioner Essentials

4 horas

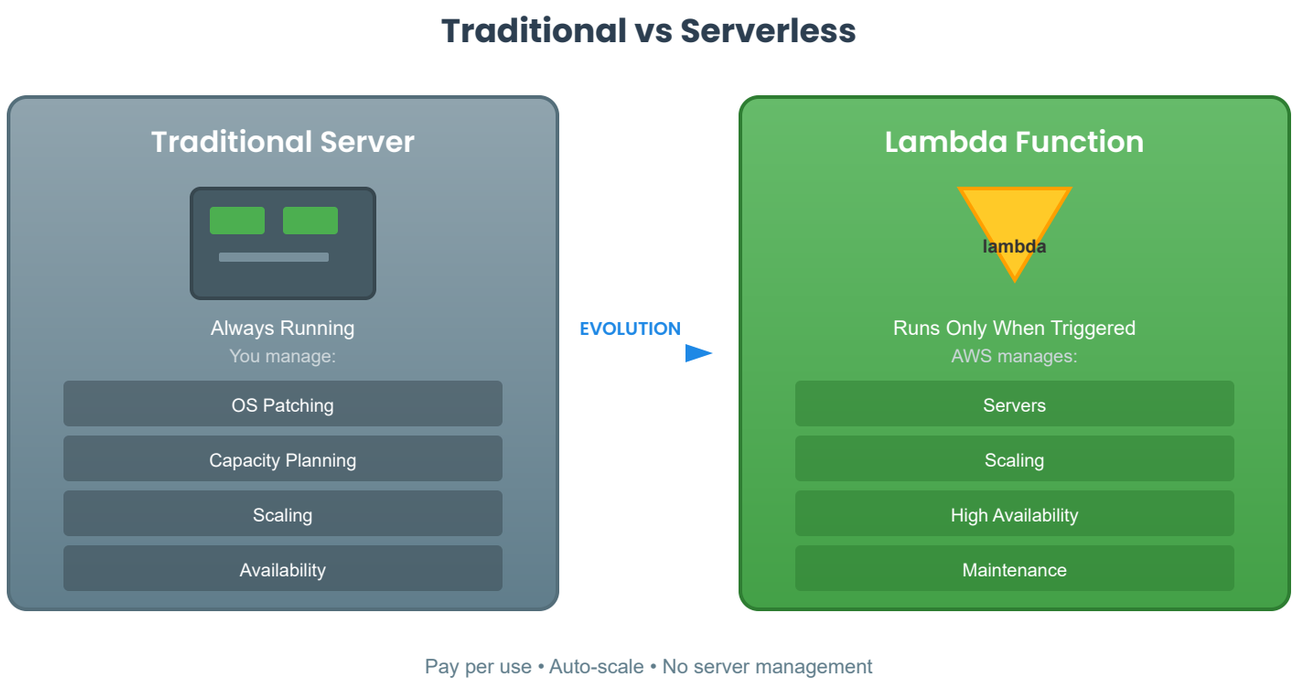
Objetivos de Aprendizaje

- Explicar el concepto de serverless computing y sus beneficios
- Implementar funciones Lambda con diferentes runtimes
- Configurar triggers y event sources para Lambda
- Desplegar APIs RESTful con Amazon API Gateway
- Crear arquitecturas event-driven con Amazon EventBridge
- Orquestar flujos de trabajo con AWS Step Functions
- Optimizar costos y rendimiento de funciones Lambda

1. Introduccion a Serverless

Conceptos fundamentales y beneficios

Traditional vs Serverless



Beneficios de Serverless

Beneficio	Descripcion
No servers to manage	Sin provisioning, patching, scaling
Continuous scaling	Auto-escala desde cero hasta miles
Pay per use	Solo paga por ejecucion
High availability	Built-in redundancy sin extra cost
Faster time to market	Menos codigo, mas enfoque en negocio

Casos de Uso Ideales

- APIs y backends web
- Procesamiento de datos (ETL)
- Automatizacion de tareas
- Event-driven architectures
- Real-time file processing
- IoT backends

Servicios Serverless en AWS

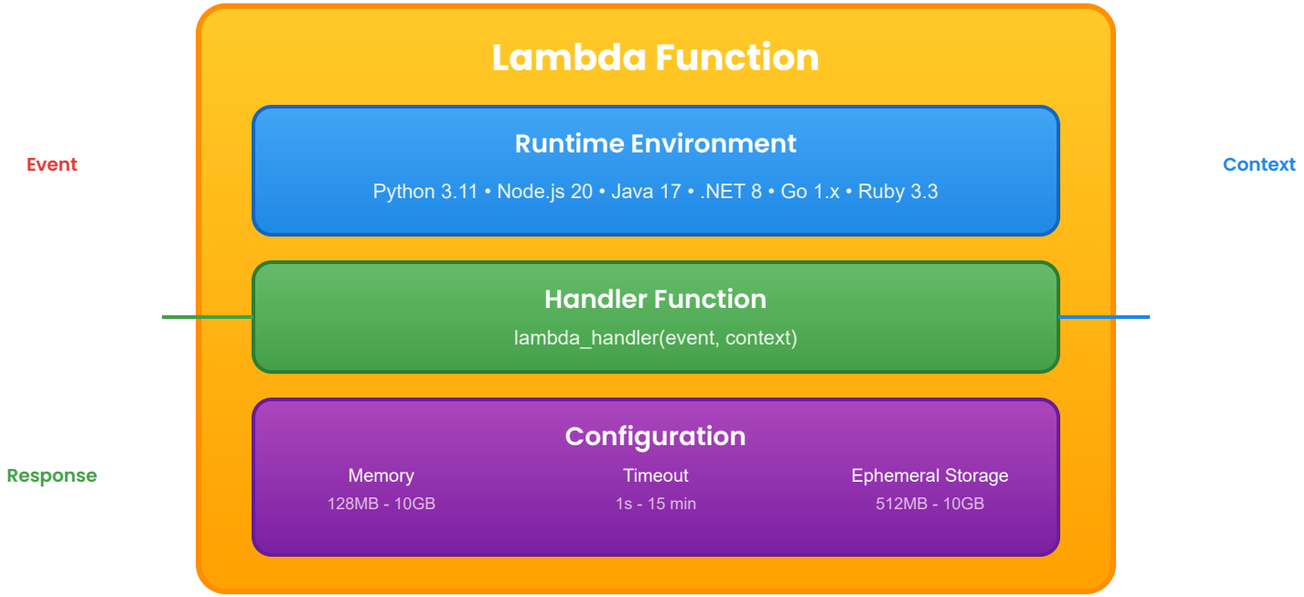
- **Computing:** Lambda, Fargate
- **API:** API Gateway, AppSync
- **Data:** DynamoDB, S3, Aurora Serverless
- **Messaging:** SNS, SQS, EventBridge
- **Orchestration:** Step Functions
- **AI/ML:** SageMaker Serverless

2. AWS Lambda

Fundamentos, runtimes y configuracion

Componentes de Lambda

Lambda Function Components



Event: JSON data from trigger • Context: Execution info (request ID, remaining time)

Runtimes Disponibles (2026)

Runtime	Versiones	SO
Python	3.8, 3.9, 3.10, 3.11, 3.12	AL2
Node.js	14, 16, 18, 20	AL2
Java	8, 11, 17, 21	AL2
.NET	6, 7, 8	AL2
Ruby	2.7, 3.2, 3.3	AL2
Go	1.x	AL2

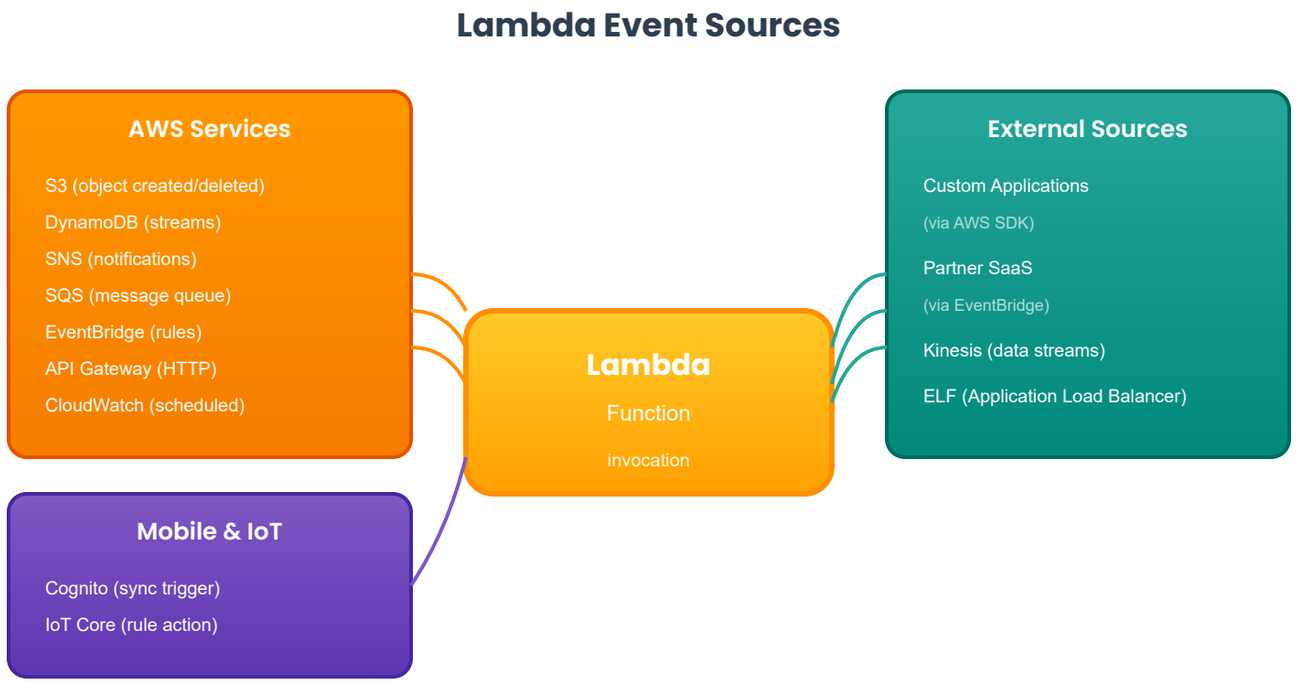
Ejemplo: Python Handler

```python
import json

def lambda_handler(event, context):
    name = event.get('name', 'World')
    message = f"Hello, {name}!"

    return {
        'statusCode': 200,
        'body': json.dumps(message),
        'headers': {'Content-Type': 'application/json'}
    }
```

Event Sources



Lambda supports synchronous and asynchronous invocation types

Configuracion de Funcion

Setting	Description	Range
Memory	RAM assigned	128MB - 10,240MB
Timeout	Max execution time	1s - 15 minutes
Ephemeral storage	/tmp space	512MB - 10,240MB
Concurrency	Simultaneous executions	0 - reserved/unlimited

Cold Starts vs Warm Starts

Cold Start: 100ms - 1s (Java/C#: 1-5s)

Warm Start: Reuse execution context, no init phase

Optimizacion de Cold Starts:

- Use smaller runtimes (Python, Node.js faster than Java)
- Minimize deployment package size
- Use Provisioned Concurrency for latency-sensitive apps
- Avoid heavy initialization in handler

Lab 6.1: API Serverless con Lambda + API Gateway

60 minutos

Pasos del Lab

1. Crear Funcion Lambda con Python 3.11
2. Crear REST API en API Gateway
3. Crear Resources y Methods (GET /users, GET /users/{id})
4. Deploy API a stage (prod)
5. Probar API con curl
6. Configurar Throttling y Quotas (opcional)

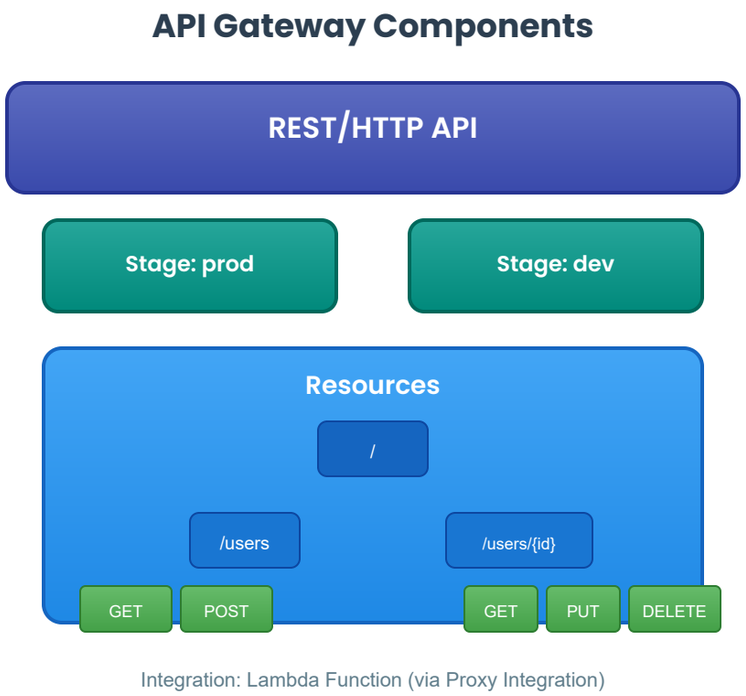
3. Amazon API Gateway

Tipos, componentes e integracion

Tipos de API

Tipo	Descripcion	Casos de uso
REST API	Full control, feature-rich	Complex APIs, legacy migration
HTTP API	Lightweight, lower cost	Serverless, simple APIs
WebSocket API	Persistent connections	Real-time apps, chat

Componentes de API Gateway



HTTP API vs REST API

Feature	HTTP API	REST API
Cost	~70% cheaper	Higher
Latency	Lower	Higher
Lambda integration	Native	Proxy required
API keys	No	Yes
Caching	No	Yes

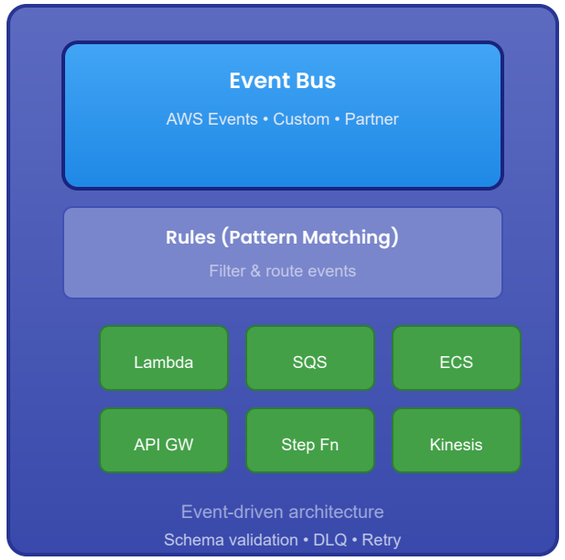
4. Amazon EventBridge

Event-driven architecture

EventBridge vs SNS

SNS (Simple Notification Service)

EventBridge



Event Bus Types

Type	Description	Use case
Default	Created automatically	AWS service events
Custom	User-created	Your applications
Partner	SaaS partner events	Third-party integrations

Estructura de un Evento

```
{
  "version": "0",
  "id": "6a7e8e2b-...",
  "detail-type": "OrderCreated",
  "source": "com.mycompany.orders",
  "account": "123456789012",
  "time": "2026-03-31T10:00:00Z",
  "region": "us-east-1",
  "detail": {
    "orderId": "ORD-12345",
    "customerId": "CUST-67890",
    "amount": 150.00,
    "status": "pending"
  }
}
```

Retry y DLQ

Retry Behavior:

- Exponential backoff: 1s, 2s, 4s, 8s... hasta 1 hora
- Dead Letter Queue (DLQ) para eventos fallidos

Lab 6.2: Pipeline de Eventos con EventBridge

45 minutos

Pasos del Lab

1. Crear SQS Queues (principal y DLQ)
2. Crear Lambda para procesar ordenes
3. Crear EventBridge Event Bus
4. Crear EventBridge Rule con pattern
5. Simular eventos con CLI
6. Verificar procesamiento en CloudWatch

5. AWS Step Functions

Orquestacion de microservicios

Que es Step Functions?

- Orquestacion de microservicios y workflows
- State machines definidas en ASL (Amazon States Language)
- Visual workflow en console
- Manejo automatico de errores y retry

Tipos de Estados

Estado	Descripcion
Task	Single unit of work (Lambda, ECS, etc.)
Choice	Branch based on condition
Parallel	Execute multiple branches simultaneously
Map	Iterate over array of items
Wait	Delay execution
Succeed/Fail	End execution success/failure

Standard vs Express Workflows

Caracteristica	Standard	Express
Duracion max	1 ano	5 minutos
Start rate	2,000/seg	100,000/seg
Coste	$0.025/1K trans	$1.00/1M exec
Sync/Async	Solo async	Sync y async

Resumen

Puntos Clave

Concepto	Puntos clave
Serverless	No servers, auto-scale, pay per use
Lambda	Runtimes, handlers, event sources, cold starts
API Gateway	REST vs HTTP API, stages, Lambda proxy
EventBridge	Event bus, rules, pattern matching
Step Functions	State machines, ASL, orchestration

Referencias

Recursos Adicionales

- AWS Lambda Documentation: https://docs.aws.amazon.com/lambda/
- API Gateway Documentation: https://docs.aws.amazon.com/apigateway/
- EventBridge Documentation: https://docs.aws.amazon.com/eventbridge/
- Step Functions Documentation: https://docs.aws.amazon.com/step-functions/
- AWS Serverless Land: https://serverlessland.com/

Gracias!

Modulo 6 Completado

Siguiente: Modulo 7 - Containers y ECS